

Análise Comparativa de Desempenho entre Busca Sequencial e Tabela Hash em Conjuntos de Dados de Grande Volume

Alanna Tomaz¹, Sofia Dios¹

¹Curso de Ciência da Computação – Centro Universitário Instituto de Educação Superior de Brasília (IESB)
Brasília – DF – Brasil

Trabalho desenvolvido para a disciplina Projeto Integrador, sob orientação do Prof. José Reginaldo de Sousa Mendes Junior.

Resumo

Este trabalho compara o desempenho de dois métodos de busca — busca sequencial e tabela hash com tratamento de colisões por encadeamento separado — em um conjunto de dados real com 400.009 registros de produtos. O objetivo foi medir, com testes controlados de tempo, qual estrutura encontra elementos mais rápido, tanto elementos que existem (no início, no meio e no final do conjunto) quanto elementos que não existem. Na Fase I, a busca sequencial teve tempo médio de 0,002454 segundos por busca, confirmando a complexidade $O(n)$ esperada. Na Fase II, a tabela hash foi implementada com tamanho igual ao número de registros ($m = 400.009$), usando a função de Dobra Simples de Dois Blocos. Mesmo com uma taxa de colisão alta (cerca de 99,6% das inserções), a tabela hash teve tempo médio de busca de 0,00003033 segundos — quase 81 vezes mais rápida que a busca sequencial. Isso confirma na prática que a tabela hash é mais eficiente, mas também mostra como a escolha da função hash influencia o resultado.

Palavras-chave: estruturas de dados; tabela hash; busca sequencial; análise de desempenho; complexidade de algoritmos.

Abstract

This paper compares the performance of two search methods — sequential search and a hash table with collision handling via separate chaining — on a real dataset with 400,009 product records. The goal was to measure, with controlled time tests, which structure finds elements faster, both elements that exist (at the beginning, middle, and end of the dataset) and elements that do not exist. In Phase I, sequential search had an average time of 0.002454 seconds per search, confirming the expected $O(n)$ complexity. In Phase II, the hash table was implemented with a size equal to the number of records ($m = 400,009$), using a simple two-block folding function. Even with a high collision rate (about 99.6% of insertions), the hash table had an average search time of 0.00003033 seconds — almost 81 times faster than sequential search.

This confirms in practice that the hash table is more efficient, but it also shows how much the choice of hash function affects the result.

Keywords: data structures; hash table; sequential search; performance analysis; algorithm complexity.

1. Introdução

Buscar elementos em um conjunto de dados é uma das operações mais comuns em sistemas computacionais. Ela aparece em bancos de dados, motores de busca, sistemas de gestão e cadastros públicos, entre outros. Quanto maior o volume de dados, mais importante fica a escolha da estrutura usada para armazenar e localizar as informações, já que isso afeta diretamente o desempenho do sistema (CORMEN et al., 2012).

O método mais simples de busca é a busca sequencial, que percorre os elementos um a um até achar o item procurado ou chegar ao fim do conjunto, com complexidade $O(n)$ no pior caso. É fácil de implementar e não precisa de nenhuma organização prévia dos dados, mas fica cada vez menos eficiente conforme o volume de registros cresce, já que o tempo de busca aumenta linearmente com o tamanho do conjunto (ZIVIANI, 2011).

Uma alternativa são as estruturas de acesso direto, como a tabela hash, que conseguem localizar um elemento em tempo médio constante, $O(1)$, independente do volume de dados (TENENBAUM; LANGSAM; AUGENSTEIN, 1995). Esse ganho depende muito da qualidade da função hash usada e de como as colisões são tratadas — colisões acontecem quando dois ou mais elementos caem na mesma posição da tabela.

Este trabalho compara, na prática, o desempenho da busca sequencial e da tabela hash com tratamento de colisões por encadeamento separado, usando um conjunto de dados real com 400.009 registros de produtos. Na Fase I do trabalho, implementamos e medimos a busca sequencial, obtendo um tempo médio geral de 0,001895 segundos por busca e confirmando o comportamento $O(n)$ esperado: buscas no início do vetor foram, em média, cerca de dez vezes mais rápidas que buscas no final, e buscas por elementos inexistentes tiveram os maiores tempos em todas as rodadas, já que exigem percorrer o vetor inteiro. Esse resultado serve de linha de base (baseline) para este artigo. Na Fase II, aqui detalhada, implementamos a tabela hash, medimos o desempenho dela com o mesmo protocolo experimental e comparamos com os resultados da Fase I, pra ver se as vantagens teóricas das estruturas de acesso direto realmente se confirmam na prática.

O restante deste artigo segue assim: a Seção 2 traz a fundamentação teórica sobre busca sequencial e tabelas hash; as seções seguintes descrevem a metodologia experimental, os resultados das duas fases, a comparação formal entre as estruturas e a discussão crítica dos resultados; por fim, a última seção apresenta as conclusões do trabalho.

2. Fundamentação Teórica

2.1. Busca Sequencial

A busca sequencial é o método de busca mais básico entre as estruturas lineares. Dado um vetor (ou lista) de n elementos e uma chave de busca, o algoritmo percorre o vetor posição por posição, comparando cada elemento com a chave procurada, até encontrar o elemento ou chegar ao fim do vetor sem sucesso (CORMEN et al., 2012).

No pior caso — quando o elemento está na última posição ou não existe no conjunto — o algoritmo faz n comparações, o que dá complexidade $O(n)$. No melhor caso, quando o elemento é o primeiro do vetor, a complexidade é $O(1)$. Em média, considerando uma distribuição uniforme das posições buscadas, o número esperado de comparações é da ordem de $n/2$, então a complexidade continua sendo $O(n)$ (ZIVIANI, 2011). Como não exige nenhuma ordenação prévia dos dados, a busca sequencial pode ser usada em qualquer situação, mas fica cada vez pior conforme o volume de registros aumenta, como mostram os experimentos da Seção 4.

Neste trabalho, a busca sequencial é a implementação feita na Fase I do projeto: cada produto do conjunto de dados fica armazenado em um vetor de structs em memória, e a busca por identificador (id) é feita comparando esse campo elemento por elemento.

Os resultados da Fase I confirmam essa previsão teórica na prática. Testando o algoritmo nos mesmos 400.009 registros usados neste artigo, com 1.000 buscas por rodada divididas entre início, meio, final e elementos inexistentes do vetor, repetidas em 3 rodadas, o tempo médio geral ficou em 0,001895 segundos por busca. Também deu pra ver que o tempo cresce conforme a posição do elemento no vetor: buscas no início foram, em média, cerca de dez vezes mais rápidas que buscas no final, e o maior tempo médio sempre apareceu no bloco de elementos inexistentes, já que esse é o pior caso do algoritmo — precisa percorrer o vetor inteiro. Os resultados completos estão na Seção 4.1.

2.2. Tabela Hash

Uma tabela hash é uma estrutura de dados que associa cada chave de busca a uma posição (bucket) de um vetor, usando uma função de espalhamento, ou função hash, $h(x)$, que transforma a chave em um índice válido da tabela. Quando a função hash é boa e a tabela tem tamanho adequado ao volume de dados, é possível buscar, inserir e remover elementos em tempo médio constante, $O(1)$ — uma vantagem grande em relação à busca sequencial, que é $O(n)$ (TENENBAUM; LANGSAM; AUGENSTEIN, 1995).

Um dos principais problemas no projeto de uma tabela hash é tratar colisões, que acontecem quando duas ou mais chaves diferentes caem na mesma posição da tabela. Entre as estratégias clássicas estão o endereçamento aberto (sondagem linear, quadrática ou hashing duplo) e o encadeamento separado (separate chaining), onde cada posição da tabela guarda a cabeça de uma lista encadeada com todos os elementos daquele índice (CORMEN et al., 2012). O encadeamento separado tem um custo extra de percorrer a lista quando há colisão, mas evita o problema de agrupamento (clustering) do endereçamento aberto e facilita a remoção de elementos. No pior caso

teórico — todas as chaves caindo no mesmo bucket — a busca em uma tabela hash com encadeamento separado vira $O(n)$, igual a uma busca sequencial em lista encadeada (ZIVIANI, 2011).

Neste trabalho, a tabela hash foi implementada com tratamento de colisões por encadeamento separado, usando listas encadeadas simples em cada bucket. O tamanho da tabela (m) foi definido como igual ao número total de registros do conjunto de dados, $m = 400.009$, conforme pedido pelo protocolo experimental. A inserção de cada novo elemento é feita sempre no início da lista do bucket correspondente, operação $O(1)$, e o contador de colisões aumenta sempre que um elemento é inserido em um bucket que já tinha pelo menos um outro elemento. A implementação também trata o caso de identificadores negativos, pra garantir que o índice calculado fique sempre dentro do intervalo válido $[0, m)$: em C, o operador módulo (%) pode dar resultado negativo quando o dividendo é negativo, o que aconteceria, por exemplo, ao buscar o identificador sentinela -999, usado no protocolo experimental para representar elementos inexistentes.

A função hash usada pelo grupo foi a função de Dobra Simples de Dois Blocos (folding), definida pela Equação 1:

$$h(x) = ((x \text{ div } 1000) + (x \text{ mod } 1000)) \text{ mod } m \quad (1)$$

onde x é o identificador (id) do produto e m é o número total de registros do conjunto de dados. A função divide o identificador em dois blocos — o quociente e o resto da divisão por 1000 — soma os dois blocos e aplica o módulo m , tentando assim distribuir as chaves pela tabela.

Por fim, os dados usados nos testes são carregados a partir de um arquivo CSV (comma-separated values) com os campos id, nome, categoria e valor de cada produto. Os registros são lidos do arquivo e guardados primeiro em um vetor em memória (a mesma estrutura usada pela busca sequencial da Fase I) e depois inseridos na tabela hash, garantindo que as duas estruturas trabalhem sobre exatamente o mesmo conjunto de dados de entrada.

3. Metodologia Experimental

Os experimentos foram feitos em linguagem C, compilada com o GCC, usando o mesmo conjunto de dados nas duas fases do trabalho (dataset4.csv, 400.009 registros). As medições da Fase II foram rodadas em ambiente Windows, pelo terminal Git Bash (MINGW64), seguindo o protocolo experimental pedido no enunciado do trabalho, pra garantir que os resultados pudessem ser reproduzidos e comparados.

O protocolo de medição de tempo tem cinco etapas. Primeiro, cada estrutura de dados — o vetor da busca sequencial e a tabela hash — foi completamente carregada com os 400.009 registros antes de começar qualquer medição, pra não misturar o tempo de carga com o tempo de busca. Segundo, foram feitas 1.000 buscas consecutivas por rodada, divididas em quatro blocos de 250 buscas, cobrindo elementos existentes no

início, no meio e no final do conjunto, mais um bloco de elementos inexistentes (identificador sentinela -999). Terceiro, os identificadores sorteados para cada bloco foram salvos em um arquivo (amostras.txt) na primeira execução e reaproveitados em todas as execuções seguintes, garantindo que a busca sequencial e a tabela hash fossem testadas com exatamente os mesmos identificadores — assim a comparação não fica injusta por causa de amostras diferentes.

Quarto, o tempo de cada bloco foi medido com a função `clock()` da biblioteca `time.h`, e o tempo médio por busca foi calculado dividindo o tempo total do bloco pelo número de buscas. Por fim, todo o protocolo foi repetido em 3 rodadas dentro da mesma execução do programa, e a média geral entre as rodadas foi usada como resultado final de cada estrutura — isso ajuda a diluir variações pontuais de carga do sistema e efeitos de cache numa única medição. O total de colisões na tabela hash foi contado durante a fase de carga, somando 1 toda vez que um elemento é inserido em um bucket que já tinha algo.

Os resultados da busca sequencial (Fase I) e da tabela hash (Fase II) estão na Seção 4, e a comparação formal entre as duas estruturas, com a complexidade teórica e as observações experimentais, está na Seção 5.

4. Resultados

4.1. Resultados da Fase I — Busca Sequencial

O protocolo experimental descrito na Fase I do projeto consistiu na execução de 1.000 buscas por rodada, divididas em quatro blocos de 250 buscas cada (início, meio, final e elemento inexistente), repetidas em 3 rodadas, totalizando 3.000 buscas medidas com a função `clock()` da biblioteca `time.h`. A Tabela 1 apresenta o tempo médio de busca, em segundos, por bloco e por rodada.

Tabela 1. Tempo médio de busca (s) por bloco e rodada — Busca Sequencial

Bloco	Rodada 1	Rodada 2	Rodada 3
Início	0,000276	0,000276	0,000304
Meio	0,001428	0,001304	0,001204
Final	0,003164	0,002624	0,002512
Inexistente	0,003244	0,003444	0,002960
Média da rodada	0,002028	0,001912	0,001745

Tabela 2. Resumo final — Busca Sequencial (400.009 registros, dataset4.csv)

Métrica	Valor
Total de registros carregados	400.009
Total de buscas medidas	3.000 (3 rodadas × 1.000 buscas)
Média geral por busca	0,001895 s

Os dados confirmam o comportamento esperado de um algoritmo $O(n)$: o tempo de busca cresce mais ou menos proporcional à posição do elemento no vetor, sendo as buscas no início do conjunto cerca de dez vezes mais rápidas que as buscas no final. O bloco de elementos inexistentes teve o maior tempo médio em todas as rodadas, já que é o pior caso do algoritmo — precisa varrer o vetor inteiro sem achar nada. Dá pra notar também uma leve queda no tempo médio ao longo das rodadas (de 0,002028 s na Rodada 1 para 0,001745 s na Rodada 3), provavelmente por causa do aquecimento do cache do processador — por isso é importante repetir o experimento em várias rodadas e reportar a média final, em vez de confiar numa única medição.

4.2. Resultados da Fase II — Tabela Hash

A medição de desempenho da tabela hash seguiu o mesmo protocolo experimental da Fase I: 1.000 buscas por rodada, divididas em quatro blocos de 250 buscas (início, meio, final e elemento inexistente), repetidas em 3 rodadas, usando exatamente os mesmos identificadores de teste (arquivo amostras.txt) usados na busca sequencial, pra garantir uma comparação justa entre as duas estruturas.

Pra fazer uma comparação pareada, sob as mesmas condições de execução, a busca sequencial foi rodada de novo na mesma sessão da medição da tabela hash, na máquina usada para os testes finais da Fase II. Essa execução conjunta deu um tempo médio geral de 0,002454 segundos por busca para a busca sequencial — esse é o valor usado como referência de comparação nesta seção, já que foi medido nas mesmas condições de hardware e carga da máquina que os resultados da tabela hash. Esse valor é parecido, em ordem de grandeza, com o tempo médio de 0,001895 segundos por busca da Fase I (Seção 4.1); a diferença provavelmente vem de variações normais de carga do sistema entre uma execução e outra.

Tabela 3. Tempo médio de busca (s) por bloco e rodada — Tabela Hash

Bloco	Rodada 1	Rodada 2	Rodada 3
Início	0,00008400	0,00006800	0,00006400
Meio	0,00004800	0,00003600	0,00003600
Final	0,00000800	0,00001200	0,00000800
Inexistente	0,00000000	0,00000000	0,00000000
Média da rodada	0,00003500	0,00002900	0,00002700

Tabela 4. Resumo final — Tabela Hash (400.009 registros, dataset4.csv)

Métrica	Valor
Total de registros carregados	400.009
Total de colisões	398.611 ($\approx 99,6\%$ das inserções)
Tempo total das execuções de busca	0,091000 s
Média geral por busca	0,00003033 s

Diferente da busca sequencial, o tempo médio da tabela hash não cresce com a posição do elemento buscado: como a estrutura acessa direto o bucket calculado pela função hash, o tempo depende só do tamanho da lista encadeada daquele bucket, não da posição do elemento no conjunto de dados original. Vale notar que os tempos médios do bloco de elementos inexistentes deram 0,00000000 segundos em todas as rodadas — não porque a busca não custa nada, mas porque a resolução da função clock() da biblioteca time.h (geralmente da ordem de milissegundos no Windows) não é boa o suficiente pra medir operações de dezenas de nanossegundos, que é o caso de uma busca em um bucket vazio ou com poucos elementos.

Comparando o tempo médio geral da tabela hash (0,00003033 s) com o da busca sequencial medido na mesma execução (0,002454 s), a tabela hash teve um desempenho cerca de 80,91 vezes melhor, mesmo com 99,6% de taxa de colisão — esse resultado é discutido com mais detalhe na Seção 6.

4.3. Print da máquina rodando localmente

```
alann@AlannaTomazNOTE MINGW64 ~/Documents/pi (main)
$ ./busca data/dataset4.csv
Dataset carregado com sucesso! Total de registros: 400009

Tabela hash carregada: 400009 registros, 398611 colisoes

Amostras carregadas de 'amostras.txt'.

=====
Rodada 1
=====
Inicio      : 250 buscas | Media: 0.000232 s/busca
Meio        : 250 buscas | Media: 0.001252 s/busca
Final       : 250 buscas | Media: 0.002996 s/busca
Inexistente : 250 buscas | Media: 0.004488 s/busca
-----
Tempo total da rodada : 2.242000 s
Media por busca       : 0.002242 s

=====
Rodada 2
=====
Inicio      : 250 buscas | Media: 0.000376 s/busca
Meio        : 250 buscas | Media: 0.001884 s/busca
Final       : 250 buscas | Media: 0.003632 s/busca
Inexistente : 250 buscas | Media: 0.004256 s/busca
-----
Tempo total da rodada : 2.537000 s
Media por busca       : 0.002537 s

=====
Rodada 3
=====
Inicio      : 250 buscas | Media: 0.000352 s/busca
Meio        : 250 buscas | Media: 0.001764 s/busca
Final       : 250 buscas | Media: 0.003648 s/busca
Inexistente : 250 buscas | Media: 0.004568 s/busca
-----
Tempo total da rodada : 2.583000 s
Media por busca       : 0.002583 s

=====
Resumo Final
=====
Media de tempo - Rodada 1 : 0.002242 s/busca
Media de tempo - Rodada 2 : 0.002537 s/busca
Media de tempo - Rodada 3 : 0.002583 s/busca
Media geral              : 0.002454 s/busca
=====
```

```

=====
Rodada 1 (HASH)
=====
Início      : 250 buscas | Média: 0.00008400 s/busca
Meio       : 250 buscas | Média: 0.00004800 s/busca
Final      : 250 buscas | Média: 0.00008800 s/busca
Inexistente : 250 buscas | Média: 0.00000000 s/busca
-----
Tempo total da rodada : 0.035000 s
Média por busca       : 0.00003500 s

=====
Rodada 2 (HASH)
=====
Início      : 250 buscas | Média: 0.00006800 s/busca
Meio       : 250 buscas | Média: 0.00003600 s/busca
Final      : 250 buscas | Média: 0.00001200 s/busca
Inexistente : 250 buscas | Média: 0.00000000 s/busca
-----
Tempo total da rodada : 0.029000 s
Média por busca       : 0.00002900 s

=====
Rodada 3 (HASH)
=====
Início      : 250 buscas | Média: 0.00006400 s/busca
Meio       : 250 buscas | Média: 0.00003600 s/busca
Final      : 250 buscas | Média: 0.00008800 s/busca
Inexistente : 250 buscas | Média: 0.00000000 s/busca
-----
Tempo total da rodada : 0.027000 s
Média por busca       : 0.00002700 s

=====
Resumo Final - TABELA HASH
=====
Média de tempo - Rodada 1 : 0.00003500 s/busca
Média de tempo - Rodada 2 : 0.00002900 s/busca
Média de tempo - Rodada 3 : 0.00002700 s/busca
Média geral              : 0.00003033 s/busca
Tempo total das execucoes : 0.091000 s
Total de colisoes        : 398611

=====
Testes concluídos com sucesso.

```

5. Comparação Formal

A Tabela 5 junta a comparação formal entre a busca sequencial e a tabela hash, mostrando o tempo médio medido, a complexidade teórica de cada estrutura e as principais observações dos experimentos das Seções 4.1 e 4.2.

Tabela 5. Comparação formal entre Busca Sequencial e Tabela Hash

Métrica	Busca Sequencial	Tabela Hash
Tempo médio	0,002454 s/busca	0,00003033 s/busca
Complexidade teórica	$O(n)$	$O(1)$ amortizado (médio) / $O(n)$ no pior caso
Observações experimentais	Tempo cresce linearmente com a posição do elemento; pior caso nos elementos inexistentes.	Tempo independente da posição original; ~80,91x mais rápida, apesar de 99,6% de colisões.

A comparação mostra que, mesmo num cenário em que a função hash usada está longe do ideal — com quase todas as inserções resultando em colisão —, a tabela hash manteve uma vantagem de desempenho de quase duas ordens de grandeza sobre a busca

sequencial. Isso confirma que o tratamento de colisões por encadeamento separado é robusto o suficiente pra manter boa parte do ganho teórico da tabela hash, mesmo quando a distribuição das chaves está longe de ser uniforme. A análise crítica desse comportamento, incluindo o cálculo da eficiência real da função hash usada, está na Seção 6.

6. Discussão Crítica

6.1. Desempenho Teórico x Prático

A complexidade $O(1)$ amortizado de uma tabela hash parte do princípio de que a função de espalhamento distribui as chaves de forma mais ou menos uniforme entre os m buckets disponíveis, de forma que o fator de carga (número de elementos dividido pelo número de buckets) determine o tamanho médio das listas encadeadas. No nosso experimento isso não aconteceu: a função hash de Dobra Simples de Dois Blocos, aplicada aos identificadores de 1 a 400.009, concentrou todas as inserções em apenas 1.398 buckets distintos dos 400.009 disponíveis — ou seja, só 0,35% da tabela foi realmente usada. Como resultado, o tamanho médio das listas nos buckets ocupados ficou em torno de 286 elementos, com o maior bucket chegando a 401 elementos.

Nessas condições, a tabela hash se comporta mais como uma busca sequencial em listas de algumas centenas de elementos do que como o caso ideal $O(1)$. Mesmo assim, como 286 elementos é bem menos do que os 400.009 registros percorridos, em média, pela busca sequencial no pior caso, a tabela hash ainda levou vantagem clara — embora a aceleração medida na prática ($\approx 80,91x$) tenha sido menor do que essa razão teórica de redução do espaço de busca por bucket. Isso provavelmente acontece por causa de custos que o modelo teórico não considera, como o uso de ponteiros e alocação dinâmica de memória em cada nó da lista, além da resolução limitada da função `clock()` usada pra medir o tempo.

6.2. Eficiência da Função Hash

A baixa eficiência da função hash usada neste trabalho vem direto da fórmula: $h(x) = ((x \div 1000) + (x \bmod 1000)) \bmod m$. Para identificadores entre 1 e 400.009, o termo $(x \div 1000)$ vai no máximo de 0 a 400, e $(x \bmod 1000)$ vai de 0 a 999; a soma desses dois termos, então, nunca passa de aproximadamente 1.399 — um valor bem menor que o tamanho da tabela ($m = 400.009$). Por causa disso, o módulo final ($\bmod m$) praticamente não faz nada: ele nunca chega a ser aplicado de verdade, já que a soma sempre é menor que m . Isso faz a função hash ignorar quase toda a capacidade da tabela, mapeando as 400.009 chaves possíveis para no máximo 1.400 índices diferentes — o que explica a taxa de colisão de 99,6% (398.611 colisões) vista na Fase II.

Esse resultado mostra que o problema é da função, não do conjunto de dados: qualquer dataset com identificadores numa faixa parecida teria o mesmo efeito de concentração, não importa o tamanho da tabela escolhido. A função de Dobra Simples de Dois Blocos costuma funcionar bem quando a soma dos blocos cobre um intervalo

parecido com o tamanho da tabela — o que não foi o caso aqui, já que m era muito maior que esse intervalo.

6.3. Vantagens e Limitações Observadas

A principal vantagem que vimos foi a robustez do encadeamento separado como forma de tratar colisões: mesmo com 99,6% das inserções colidindo, a estrutura continuou funcionando direito e bem mais rápida que a busca sequencial, sem precisar reorganizar a tabela ou usar uma segunda função hash. Isso é uma vantagem importante do encadeamento separado em relação ao endereçamento aberto, que costuma piorar mais quando a taxa de colisão é alta, por causa do agrupamento (clustering) de elementos.

Por outro lado, vimos duas limitações importantes. A primeira é o desperdício de memória: dos 400.009 buckets alocados pela tabela (cada um é um ponteiro), 398.611 (99,65%) nunca foram usados, o que é bem mais memória alocada do que o necessário pra essa quantidade de dados. A segunda é a falta de previsibilidade no tempo de busca: como o tamanho das listas varia de bucket pra bucket — de listas vazias até listas com 401 elementos —, o tempo de uma busca específica pode variar bastante dependendo do identificador, mesmo que a média geral seja baixa. Essas limitações mostram que só definir o tamanho da tabela não garante o desempenho $O(1)$ esperado de uma tabela hash; a função de espalhamento escolhida é o que realmente determina se esse potencial teórico vira resultado prático.

7. Conclusão

Este trabalho teve como objetivo comparar, na prática, o desempenho da busca sequencial e da tabela hash com tratamento de colisões por encadeamento separado, na busca por registros em um conjunto de dados real com 400.009 produtos. Os resultados confirmaram que a tabela hash é melhor que a busca sequencial: o tempo médio de busca da tabela hash foi cerca de 80,91 vezes menor, mostrando na prática o ganho de desempenho previsto pela complexidade teórica $O(1)$ amortizado, mesmo com um cenário de espalhamento bem desfavorável.

Esse cenário desfavorável é, na verdade, um dos achados mais interessantes do trabalho: a função hash de Dobra Simples de Dois Blocos, aplicada aos identificadores do dataset usado (de 1 a 400.009), gerou uma taxa de colisão de aproximadamente 99,6% das inserções. Isso acontece porque a soma dos dois blocos da função (quociente e resto da divisão por 1000) nunca passa de um intervalo bem menor que o tamanho da tabela, então o operador módulo praticamente não distribui as chaves entre as posições disponíveis. Mesmo assim, a estrutura encadeada manteve um desempenho médio bem melhor que o da busca sequencial, o que mostra que o encadeamento separado é uma forma robusta de tratar colisões.

Concluindo, a escolha da função hash é determinante para a eficiência de uma tabela hash. Funções mal dimensionadas para o conjunto de chaves — como a deste experimento — atrapalham bastante o potencial teórico da estrutura, mesmo sem anulá-lo completamente quando combinadas com um bom tratamento de colisões.

Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

ZIVIANI, N. Projeto de Algoritmos: com Implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.

TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J. Estruturas de Dados Usando C. São Paulo: Pearson Makron Books, 1995.